Variational Autoencoders with Enhanced Capacity and Conditional GANs for Data Augmentation under Limited Data Regimes

(Author Name)
(Affiliation)
(Email)

January 26, 2026

Abstract

Deep generative models provide a principled framework for learning complex data distributions and have become central to modern unsupervised and semi-supervised learning. Among the most widely studied approaches are Variational Autoencoders (VAEs) and Generative Adversarial Networks (GANs), each offering complementary strengths and limitations.

VAEs are trained by maximizing a variational lower bound on the data log-likelihood and provide an explicit probabilistic model with tractable inference. However, standard VAEs are known to suffer from limited expressiveness and optimization pathologies such as posterior collapse, where the learned latent variables fail to carry meaningful information. Recent work has proposed modified objectives and hierarchical latent architectures to address these issues by increasing model capacity and encouraging effective latent utilization.

GANs, in contrast, learn to generate samples through an adversarial training process and often produce sharper samples but lack an explicit likelihood. When training data are limited, GANs offer a particularly valuable opportunity: they can be used as data generators to augment small datasets and improve the generalization performance of downstream classifiers. Nevertheless, GAN training is notoriously unstable, motivating the development of techniques such as Wasserstein objectives, gradient penalties, spectral normalization, and attention mechanisms.

This report investigates both paradigms in a unified experimental framework. First, we analyze standard and modified VAEs on Dynamic MNIST, focusing on likelihood estimation, latent structure, and posterior behavior. Second, we design a conditional WGAN-GP with attention mechanisms for FashionMNIST and evaluate its effectiveness as a data augmentation tool under constrained data conditions.

Contents

1 Introduction

1.1 Problem Statement

In this report, we address the problem of template-to-photo alignment, where the goal is to geometrically align a 2D reference template image, denoted as $\mathcal{I}_T$, to a photograph $\mathcal{I}_Q$ of the corresponding physical part. The reference template $\mathcal{I}_T$ could represent any annotated 2D object (e.g., a CAD-derived drawing, a part map, or a blueprint), while $\mathcal{I}_Q$ is the photograph that contains the physical part whose geometric correspondence with the template we need to find.

The central challenge lies in computing an accurate geometric mapping between the two images, using a technique that can robustly handle differences in viewpoint, illumination, and background clutter. The final output is a visual overlay that maps the template onto the photograph, enabling visual inspection and/or facilitating downstream measurement tasks such as part alignment, dimensioning, or quality control.

1.2 Motivation

Template-to-photo alignment is a fundamental task in various industrial and engineering applications. For example, in automated quality control, part inspection, and assembly verification, being able to precisely align digital templates (e.g., CAD files or part blueprints) with real-world photographs enables automated systems to assess whether physical components match their specifications. However, this task is complicated by several real-world factors such as changes in viewpoint, lighting conditions, and cluttered backgrounds.

A practical solution to this problem must achieve the following:

- **Robustness to variation:** The system should handle moderate changes in viewpoint, illumination variation, and background clutter. It should also cope with partial occlusions and distortions that are common in real-world environments.
- **Accuracy and precision:** The alignment must be precise enough to support downstream measurement tasks, such as detecting dimensional deviations or ensuring the part fits within specified tolerances.
- **Compute efficiency:** The method must operate efficiently under constrained compute resources. For industrial deployment, it is important that the system can run in real-time or near real-time, with minimal computational overhead.
- **Measurable performance metrics:** The system should provide quantifiable metrics such as alignment error and success rate, allowing for objective performance evaluation.

1.3 Contributions

This work presents a novel, feature-based alignment method that satisfies the aforementioned requirements and demonstrates its applicability in template-to-photo alignment tasks. The contributions of this work are as follows:

- **A complete, modular implementation:** We provide a full implementation of the alignment pipeline with well-defined stages and outputs. The system is built with reproducibility in mind, ensuring that all results can be independently verified.
- **A robust alignment method:** The proposed method leverages feature matching and homography estimation to compute the geometric transformation between the template and photograph. It is designed to be robust to noise, variations in lighting, and partial occlusions.

- **Evaluation protocol:** We define a rigorous evaluation protocol with both quantitative and qualitative analyses. We use well-established metrics such as mean/median reprojection error and success rate, and we provide failure-case diagnostics to analyze the limits of the system.
- **Reproducibility artifacts:** All results, including metrics CSVs and generated figures, are available for download, enabling others to reproduce the reported findings and build upon this work. The full source code and dataset split details are included for transparency.

This report not only contributes to the field of image alignment but also provides insights into the challenges and solutions that arise when aligning template images to real-world photographs under varying conditions.

2 Notation and Definitions

This section defines symbols and terminology used throughout the report for both components: (i) VAEs (baseline and the referenced enhanced-capacity/hierarchical variant) on Dynamic MNIST, and (ii) conditional WGAN-GP for data augmentation on FashionMNIST.

2.1 Symbols

Data and distributions (VAE).

- $x \in \{0,1\}^D$: observed image (Dynamic MNIST uses dynamic binarization; $D = 28 \times 28$).
- $z \in \mathbb{R}^d$: continuous latent variable; d is the latent dimensionality.
- $p(z)$: latent prior (baseline uses $p(z) = \mathcal{N}(0, I)$).
- $q_\phi(z \mid x)$: encoder / variational posterior parameterized by ϕ.
- $p_\theta(x \mid z)$: decoder / likelihood parameterized by θ.
- $\mathcal{L}_{\text{ELBO}}(x)$: evidence lower bound (ELBO).
- $\text{KL}(q\|p)$: Kullback–Leibler divergence.
- $\mathcal{L}_{\text{rec}}(x) = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)]$: reconstruction term (negative expected log-likelihood).
- $\mathcal{L}_{\text{KL}}(x) = \text{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$: KL regularizer term (baseline).

Enhanced-capacity prior / pseudo-inputs (referenced VAE).

- $\{u_k\}_{k=1}^K$: pseudo-inputs (learnable parameters in input space or outputs of a pseudo-input network).
- $p_\lambda(z) = \frac{1}{K}\sum_{k=1}^K q_\phi(z \mid u_k)$: learned mixture prior with parameters λ (through $\{u_k\}$).
- $\mathcal{L}_{\text{KL}}^{(\text{mix})}(x) = \text{KL}\big(q_\phi(z \mid x) \,\|\, p_\lambda(z)\big)$: KL term against the learned prior.

Hierarchical VAE (two-layer latent).

- $z_2 \in \mathbb{R}^{d_2}$: top-level latent variable.
- $z_1 \in \mathbb{R}^{d_1}$: bottom-level latent variable.
- $p_\theta(z_1 \mid z_2)$: conditional prior (top-down) in the hierarchical decoder.
- $q_\phi(z_2 \mid x)$, $q_\phi(z_1 \mid x, z_2)$: hierarchical inference factors.

Likelihood estimation.

- K_{IS}: number of importance samples for likelihood estimation.
- $\widehat{\log p_\theta(x)}$: importance-weighted estimate of log-likelihood (IWAE-style).

Conditional WGAN-GP (GAN augmentation).

- $y \in \{1, \ldots, C\}$: class label ($C = 10$ for FashionMNIST).
- $z \sim \mathcal{N}(0, I)$: generator noise vector.
- $G_\psi(z, y)$: conditional generator parameterized by ψ.
- $D_\omega(x, y)$: conditional critic/discriminator parameterized by ω (projection discriminator).
- $e_G(y) \in \mathbb{R}^{d_y}$: generator label embedding; $e_D(y) \in \mathbb{R}^{d_f}$: critic label embedding.
- $\phi_\omega(x) \in \mathbb{R}^{d_f}$: critic feature representation used in projection discrimination.
- λ_{GP}: gradient penalty coefficient (assignment specifies $\lambda_{\mathrm{GP}} = 10$).
- $\epsilon_{\mathrm{drift}}$: drift penalty coefficient (assignment specifies 0.001).
- λ_{emb}: embedding L_2 regularization coefficient (assignment specifies 0.001).
- N_{critic}: number of critic updates per generator update (assignment specifies 5).
- RUN: total training steps (assignment specifies 4000).

2.2 Abbreviations

- VAE: Variational Autoencoder.
- ELBO: Evidence Lower Bound.
- KL: Kullback–Leibler divergence.
- IWAE: Importance Weighted Autoencoder estimator (importance sampling for likelihood).
- cGAN: Conditional GAN.
- WGAN-GP: Wasserstein GAN with Gradient Penalty.
- SN: Spectral Normalization.
- SA: Self-Attention (spatial attention).
- CA: Channel Attention (Squeeze-and-Excitation style).
- PD: Projection Discriminator.

2.3 Key Modeling Assumptions

- **Dynamic MNIST observation model:** pixels are modeled as conditionally independent Bernoulli variables given latents ($p_\theta(x \mid z)$ Bernoulli), consistent with the dynamic binarization protocol.
- **Variational inference:** $q_\phi(z \mid x)$ (and its hierarchical factors) are Gaussian with diagonal covariance, enabling reparameterization-based training.
- **WGAN-GP Lipschitz control:** the critic is approximately 1-Lipschitz via gradient penalty (and SN on selected layers), which is required for the Wasserstein objective to be well-behaved.
- **Augmentation validity:** synthetic samples are used only to expand the training distribution; generalization is judged exclusively on held-out real test data.

3 System Overview

3.1 Pipeline Summary

The alignment system described in this report follows a standard feature-based registration pipeline, which processes the template and photograph in a sequence of deterministic stages to achieve the desired template-to-photo overlay. The overall steps are as follows:

1. **Preprocessing of Images:** Both the template image ($\mathcal{I}_T$) and the query photograph ($\mathcal{I}_Q$) are preprocessed. This typically involves converting the images to grayscale for simplicity and ensuring they are normalized for consistent feature extraction. In some cases, contrast normalization (e.g., CLAHE) is applied to enhance keypoint detectability, especially when dealing with low-contrast images.
2. **Keypoint Detection and Description:** Local features are detected using the Oriented FAST and Rotated BRIEF (ORB) algorithm (**?**). ORB is chosen for its computational efficiency and its robustness to rotation and scale variations in the images. The detected keypoints are described using binary descriptors, which are efficient for matching and computationally lightweight.
3. **Descriptor Matching and Filtering:** Once the descriptors are extracted, they are matched between the template and the query image. Hamming distance is used to measure the similarity between descriptors. A cross-checking procedure is applied to reduce incorrect matches, and a distance threshold is used to filter out weak matches, ensuring only the most reliable correspondences are retained.
4. **Homography Estimation via RANSAC:** The next step is to estimate the homography $\mathbf{H}$, which models the projective transformation between the template and the query photograph. The homography is computed using RANSAC (**?**), a robust estimator that minimizes the effect of outlier matches by iteratively fitting the model to the data and retaining only the inliers that are consistent with the estimated homography.
5. **Template Warping and Overlay Generation:** After estimating the homography, the template is warped into the coordinate frame of the photograph using the computed homography $\mathbf{H}$. The warped template is then blended with the query photograph to create the final overlay image for inspection.
6. **Metrics Export and Analysis:** Quantitative performance metrics, such as reprojection error, success rate, and runtime, are computed and exported. Additionally, failure-case

examples are identified and stored to provide insights into boundary conditions and potential areas for improvement in the system.

3.2 Repository Layout (Expected)

To maintain organization and clarity, the repository is structured with clear directory divisions for each component. Below is the expected folder layout, which helps manage data, scripts, and results:

```
report/
  main.tex          % Main report LaTeX file
  *.tex             % Other LaTeX files (e.g., for individual sections)
  references.bib    % BibTeX file containing references
src/
  (your python modules) % Python scripts for the alignment pipeline
data/
  templates/        % Folder containing the template images
  photos/           % Folder containing the query photographs
  annotations/      % (optional) Folder for ground-truth point correspondences or masks
results/
  overlays/         % Folder for saving overlay images (template overlaid on query images)
  figures/          % Folder for figures used in the report (e.g., error distributions, runt
  metrics/          % Folder for quantitative metrics (CSV/JSON files)
  matches/          % (optional) Folder for debug outputs showing raw matches or intermediat
```

This structure ensures that each component of the project—data, code, and results—are clearly separated, facilitating both reproducibility and further experimentation.

3.3 Why Feature-Based Registration

Feature-based registration is a widely adopted method for aligning images, particularly when dealing with planar scenes or objects where the relationship between the template and the query image can be approximated by a projective transformation (homography). This approach has several advantages:

- **Computational Efficiency:** Feature-based methods like ORB provide a good balance between accuracy and computational efficiency. ORB's binary descriptors are lightweight and allow for fast matching, which is essential for real-time applications or large-scale datasets.
- **No Training Data Required:** Unlike deep learning-based methods, feature-based registration does not require large datasets for training. This makes it a particularly appealing choice when labeled data is scarce or when the method needs to be applied to a wide variety of images without re-training.
- **Robustness to Moderate Viewpoint Changes:** ORB features are invariant to rotation and scale, making the method robust to moderate viewpoint changes and ensuring that keypoints remain detectable even when the template and photograph are viewed from different angles or distances.
- **Low Latency:** The method operates with low latency, making it suitable for applications that require fast processing, such as interactive inspection systems or on-the-fly part verification during assembly lines.

Feature-based registration methods are particularly effective when there is sufficient texture or structure in the images for feature detection. Under moderate viewpoint changes, these methods can yield high-quality alignments with low computational overhead (**??**). However, they may struggle in the presence of repetitive textures, glare, or occlusion, which can be mitigated by applying appropriate preprocessing and matching strategies.

By focusing on feature-based methods, this work provides a solution that is both efficient and flexible, making it applicable to a wide range of practical scenarios in industrial inspection, robotic assembly, and other areas requiring template-to-photo alignment.

4 Related Work

Generative modeling has two dominant paradigms relevant to this report: likelihood-based latent-variable models (VAEs) and adversarial models (GANs). Both have extensive literature on objectives, stability, and failure modes, particularly under limited data or highly expressive decoders.

4.1 Variational Autoencoders and objective design

VAEs optimize a variational lower bound on $\log p_\theta(x)$, enabling scalable amortized inference via an encoder network (**??**). While VAEs provide a principled likelihood-based framework, empirical work has documented limitations including blurry samples (under simple likelihoods) and *posterior collapse*, where the KL term approaches zero and the latent variable becomes uninformative, especially with expressive decoders.

A major line of research improves VAE performance by changing the objective or increasing representational capacity. Importance-weighted objectives provide tighter bounds and can improve training and evaluation by using multiple importance samples (**?**). Other approaches alter priors (e.g., richer latent distributions) or introduce hierarchical latents, enabling multimodal structure and better latent utilization. The referenced method in this assignment follows this direction by introducing a modified loss/prior mechanism and a hierarchical architecture, with the explicit goal of increasing capacity and mitigating collapse-like behavior.

4.2 Likelihood estimation for latent-variable models

Exact likelihood computation for latent-variable models is generally intractable due to the integral over z. Importance sampling (IWAE-style estimators) is a standard approach for estimating $\log p_\theta(x)$ in a controlled and comparable way across models (**?**). This report uses a consistent estimator and sample budget across VAE variants to ensure fair comparisons.

4.3 Conditional GANs, stability, and augmentation

GANs generate samples by adversarial training and often yield sharper samples than likelihood-based models, but training can be unstable and sensitive to architecture and regularization. Wasserstein GAN objectives improve gradient behavior by approximating Earth Mover distance, and WGAN-GP enforces the Lipschitz constraint via a gradient penalty, substantially improving stability in practice (**?**). Spectral normalization further stabilizes training by constraining the discriminator's operator norm (**?**).

For conditional generation, projection discriminators incorporate label conditioning through inner products in feature space, improving class-conditional fidelity and stability relative to naive concatenation (**?**). Attention mechanisms have also proven effective in generative modeling by enabling long-range spatial interactions (self-attention) and adaptive channel reweighting (channel attention, commonly formalized as squeeze-and-excitation) (**??**).

Finally, data augmentation is a standard remedy for limited training data in supervised learning. In this report, GAN-based augmentation is evaluated by a controlled comparison between classifiers trained on the same limited real subset with and without synthetic samples, with all other training factors held fixed.

5 Methodology

5.1 Variational Autoencoders (VAE)

Latent-variable model and variational inference. We consider a latent-variable generative model

$$p_\theta(x, z) = p_\theta(x \mid z)\, p(z),$$

where $x \in \mathbb{R}^D$ denotes an observed image and $z \in \mathbb{R}^d$ is a continuous latent variable. Since the exact posterior $p_\theta(z \mid x)$ is typically intractable, we introduce an amortized variational approximation (encoder)

$$q_\phi(z \mid x).$$

For Dynamic MNIST, we use a Bernoulli observation model

$$p_\theta(x \mid z) = \prod_{i=1}^{D} \mathrm{Bernoulli}\big(x_i; \pi_\theta(z)_i\big),$$

where $\pi_\theta(z) \in (0,1)^D$ is the decoder output after a sigmoid nonlinearity.

ELBO objective and KL regularization. Training proceeds by maximizing the evidence lower bound (ELBO) on the log-likelihood $\log p_\theta(x)$ (**??**):

$$\log p_\theta(x) \;\geq\; \mathcal{L}_{\mathrm{ELBO}}(x) = \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x \mid z)\right] - \mathrm{KL}(q_\phi(z \mid x) \,\|\, p(z))\,.$$

The first term is a reconstruction (negative cross-entropy for Bernoulli pixels), while the KL divergence regularizes the approximate posterior toward the prior, controlling latent capacity and enabling sampling from $p(z)$ at test time.

Reparameterization trick. We parameterize $q_\phi(z \mid x)$ as a diagonal Gaussian,

$$q_\phi(z \mid x) = \mathcal{N}\big(z; \mu_\phi(x), \mathrm{diag}(\sigma_\phi^2(x))\big)\,,$$

and apply the reparameterization trick to enable low-variance gradient estimation:

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

Enhanced-capacity objective: flexible prior via pseudo-inputs. A core limitation of standard VAEs is that a simple fixed prior (e.g., $\mathcal{N}(0, I)$) can over-regularize $q_\phi(z \mid x)$, encouraging inactive latent dimensions and contributing to posterior collapse, particularly when the decoder is expressive. Following the referenced method, we replace the fixed prior with a learnable, data-adaptive prior defined through *pseudo-inputs* $\{u_k\}_{k=1}^K$:

$$p_\lambda(z) = \frac{1}{K} \sum_{k=1}^{K} q_\phi(z \mid u_k),$$

where u_k are optimized parameters in input space (or are generated by a small network) and $q_\phi(z \mid u_k)$ reuses the encoder. This prior increases representational capacity by forming a

multimodal latent distribution aligned with the aggregate posterior, thereby relaxing an overly restrictive KL constraint. The training objective becomes:

$$\mathcal{L}(x) = \mathbb{E}_{q_\phi(z|x)} \left[\log p_\theta(x \mid z)\right] - \mathrm{KL}(q_\phi(z \mid x) \,\|\, p_\lambda(z)) \,,$$

with the KL now computed against the learned mixture prior. In practice, this modification can improve latent utilization and sample quality by avoiding a mismatch between $q_\phi(z \mid x)$ and a simplistic prior.

Hierarchical latent architecture. To further increase capacity, we implement a hierarchical latent model with two stochastic layers (e.g., $z_2 \to z_1 \to x$), where the generative model factorizes as

$$p_\theta(x, z_1, z_2) = p_\theta(x \mid z_1)\, p_\theta(z_1 \mid z_2)\, p(z_2),$$

and the inference model factorizes as

$$q_\phi(z_1, z_2 \mid x) = q_\phi(z_2 \mid x)\, q_\phi(z_1 \mid x, z_2).$$

The corresponding ELBO is

$$\mathcal{L}_{\text{hier}}(x) = \mathbb{E}_{q_\phi(z_1,z_2|x)} \left[\log p_\theta(x \mid z_1)\right] - \mathbb{E}_{q_\phi(z_2|x)} \left[\mathrm{KL}(q_\phi(z_1 \mid x, z_2) \,\|\, p_\theta(z_1 \mid z_2))\right] - \mathrm{KL}(q_\phi(z_2 \mid x) \,\|\, p(z_2)) \,,$$

where the top-level prior $p(z_2)$ may be chosen as a standard normal or replaced by the pseudo-input prior $p_\lambda(z_2)$, depending on the assignment's specification.

Likelihood estimation. Since $\log p_\theta(x) = \log \int p_\theta(x \mid z) p(z)\, dz$ is not directly tractable, we estimate log-likelihood using an importance-weighted (IWAE-style) estimator (**?**). For K importance samples $z^{(k)} \sim q_\phi(z \mid x)$,

$$\log p_\theta(x) \approx \log \left(\frac{1}{K} \sum_{k=1}^{K} \frac{p_\theta(x \mid z^{(k)})\, p(z^{(k)})}{q_\phi(z^{(k)} \mid x)} \right).$$

Diagnostics for posterior behavior and collapse. To analyze latent utilization, we report (i) KL contributions per dimension (or per group in hierarchical models), (ii) active units (e.g., dimensions with $\mathrm{Var}_x[\mathbb{E}(z \mid x)]$ above a threshold), and (iii) aggregate posterior statistics compared with the prior. We complement these with latent traversals and PCA of inferred codes to test whether principal directions correspond to meaningful factors of variation.

5.2 Conditional GAN for Data Augmentation (WGAN-GP)

Problem setting and data splits. We consider FashionMNIST under a limited-data regime. The training set is partitioned into two *non-overlapping* subsets: one for GAN training and one for classifier training. The GAN is used to generate additional labeled samples for augmentation; the downstream classifier is trained on either real-only or real+synthetic data and evaluated on a held-out test set.

Conditional generator. The generator is conditional on class label $y \in \{1, \ldots, C\}$ and noise $z \sim \mathcal{N}(0, I)$. A learnable embedding $e_G(y) \in \mathbb{R}^{d_y}$ is concatenated with z to form the generator input:

$$\tilde{z} = [z;\, e_G(y)], \qquad x_{\text{fake}} = G_\psi(\tilde{z}, y).$$

The architecture follows the assignment's specified ConvTranspose/upsampling blocks, including attention modules at designated resolutions.

Projection discriminator (conditional critic). The discriminator (critic) follows the projection discriminator formulation (**?**). Let $\phi_\omega(x) \in \mathbb{R}^{d_f}$ denote the discriminator feature vector before the final output layer, and let $e_D(y) \in \mathbb{R}^{d_f}$ be the label embedding in the discriminator. The conditional score is computed as

$$D_\omega(x, y) = s_\omega(\phi_\omega(x)) + \langle \phi_\omega(x),\, e_D(y) \rangle,$$

where $s_\omega(\cdot)$ is a learned scalar head and $\langle \cdot, \cdot \rangle$ denotes an inner product. This construction enforces class-conditional discrimination without explicitly concatenating labels to image channels.

Spectral normalization. To stabilize adversarial training, we apply spectral normalization (SN) to convolutional (and transposed convolutional) layers (**?**). SN constrains each layer's operator norm by normalizing its weight matrix W via its largest singular value $\sigma_{\max}(W)$:

$$\bar{W} = \frac{W}{\sigma_{\max}(W)},$$

which controls the Lipschitz constant of the discriminator and improves training stability.

Attention mechanisms. We incorporate two attention modules, following the assignment definitions.

Self-attention (as in SAGAN (**?**)) captures long-range spatial dependencies. Given feature maps $x \in \mathbb{R}^{B \times C \times H \times W}$ with $N = HW$, we compute projections f, g, h and attention weights $\beta \in \mathbb{R}^{B \times N \times N}$:

$$\beta = \mathrm{softmax}(f^\top g),$$

and produce an output o aggregated from h (and then mapped through v), followed by a residual connection with a learnable scalar γ:

$$y = \gamma o + x.$$

Channel attention (SE-style (**?**)) recalibrates channel-wise responses. Global average pooling produces a channel descriptor $c \in \mathbb{R}^{B \times C}$, which is passed through two fully-connected layers with ReLU and a sigmoid gate to generate channel weights that modulate the original feature map.

WGAN-GP objective with regularization. We use the Wasserstein GAN objective with gradient penalty (WGAN-GP) (**?**). The critic aims to maximize the Wasserstein estimate:

$$\mathcal{W} \approx \mathbb{E}[D(x_{\text{real}}, y)] - \mathbb{E}[D(x_{\text{fake}}, y)],$$

subject to a soft 1-Lipschitz constraint enforced by the gradient penalty:

$$\mathcal{L}_{\text{GP}} = \lambda_{\text{GP}} \mathbb{E}_{\hat{x}} \left(\|\nabla_{\hat{x}} D(\hat{x}, y)\|_2 - 1 \right)^2, \qquad \lambda_{\text{GP}} = 10,$$

where $\hat{x} = \epsilon x_{\text{real}} + (1 - \epsilon) x_{\text{fake}}$ and $\epsilon \sim \mathcal{U}(0, 1)$.

In addition, we incorporate: (i) a drift penalty to prevent critic outputs from drifting:

$$\mathcal{L}_{\text{drift}} = \epsilon_{\text{drift}}\, \mathbb{E}\left[D(x_{\text{real}}, y)^2 \right], \qquad \epsilon_{\text{drift}} = 0.001,$$

and (ii) discriminator embedding regularization:

$$\mathcal{L}_{\text{emb}} = \lambda_{\text{emb}}\, \|e_D\|_2^2, \qquad \lambda_{\text{emb}} = 0.001.$$

The discriminator loss (to minimize) is therefore:

$$\mathcal{L}_D = -\Big(\mathbb{E}[D(x_{\text{real}}, y)] - \mathbb{E}[D(x_{\text{fake}}, y)] \Big) + \mathcal{L}_{\text{GP}} + \mathcal{L}_{\text{drift}} + \mathcal{L}_{\text{emb}},$$

and the generator loss is:

$$\mathcal{L}_G = -\mathbb{E}[D(x_{\text{fake}}, y)].$$

Optimization and training protocol. We use Adam optimizers with the assignment's hyperparameters: learning rates $\mathrm{LR}(G) = 2 \times 10^{-4}$ and $\mathrm{LR}(D) = 4 \times 10^{-4}$, $\beta_1 = 0.0$, $\beta_2 = 0.9$, and an exponential decay scheduler with $\gamma = 0.95$ applied every 100 steps. Weight decay is grouped such that embedding parameters receive decay 10^{-3} and other parameters receive zero decay, matching the specification. We update the discriminator $N_{\text{critic}} = 5$ times per generator update, detach fake samples during critic updates, and log the Wasserstein estimate, gradient penalty, drift, embedding regularization, and total losses at each step.

Sampling protocol and augmentation evaluation. To monitor training progression, we generate class-conditional grids at fixed steps using fixed noise and fixed labels (covering all classes), enabling controlled qualitative comparisons across time. For augmentation, we generate a specified number of synthetic samples per class and train a classifier under two regimes: real-only and real+synthetic. We then compare overall accuracy, class-wise performance, and error patterns to quantify whether generative augmentation improves generalization in the limited data setting.

6 Data and Experimental Protocol

6.1 Datasets

This report uses two standard vision datasets selected to match the two tasks: (i) likelihood-based generative modeling and latent analysis (Dynamic MNIST), and (ii) conditional generation for augmentation with limited labeled data (FashionMNIST).

6.1.1 Dynamic MNIST

Dynamic MNIST is derived from the MNIST handwritten digits dataset (28×28 grayscale). Instead of using a single fixed binarization, each image is binarized *stochastically* at each presentation using the pixel intensity as a Bernoulli parameter:

$$x_i \sim \mathrm{Bernoulli}(x_i^{\mathrm{gray}}), \quad i = 1, \ldots, D.$$

This yields a distribution over binary images that more faithfully reflects uncertainty in pixel values and is widely used in likelihood-based generative modeling studies. The dataset is used to evaluate:

- baseline VAE vs proposed VAE objective/architecture,
- low-dimensional vs high-dimensional latent settings,
- posterior behavior, collapse indicators, and likelihood estimates.

6.1.2 FashionMNIST

FashionMNIST contains 28×28 grayscale images across 10 clothing categories (e.g., T-shirt/top, trouser, pullover). It is used for conditional image generation and augmentation. Images are normalized to $[-1, 1]$ using:

$$x \leftarrow \frac{x - 0.5}{0.5}.$$

FashionMNIST supports clean class-conditional evaluation and direct measurement of augmentation effects on classifier generalization.

6.2 Limited-data regime and non-overlapping splits (FashionMNIST)

To simulate limited labeled data and to ensure the augmentation experiment is scientifically valid, we construct two *non-overlapping* subsets of the FashionMNIST training set:

- $\mathcal{D}_{\text{GAN}}$: used exclusively to train the conditional WGAN-GP generator/critic.
- $\mathcal{D}_{\text{CLS}}$: used exclusively to train the downstream classifier.

The subsets are stratified by class, and the per-class sample counts follow the assignment specification (e.g., N samples per class). The index lists of both subsets are saved to disk (e.g., `data/splits/`) and reused for all runs. This prevents accidental overlap and eliminates a key confounder: if $\mathcal{D}_{\text{GAN}} \cap \mathcal{D}_{\text{CLS}} \neq \emptyset$, the generator can effectively memorize classifier training examples, and "augmentation" would not be an independent distributional expansion.

6.3 Train/Validation/Test protocol

VAE (Dynamic MNIST). Models are trained on the training split. Evaluation uses a held-out test split for: (i) likelihood estimation (importance-weighted estimator), (ii) latent diagnostics (posterior statistics and PCA/traversals), (iii) qualitative sample generation. If a validation set is used for hyperparameter selection (e.g., learning rates, architecture depth), it is formed only from the training split to preserve test integrity.

GAN augmentation (FashionMNIST).

- The GAN is trained only on $\mathcal{D}_{\text{GAN}}$.
- The classifier is trained only on $\mathcal{D}_{\text{CLS}}$ (real-only baseline) or on $\mathcal{D}_{\text{CLS}}$ plus synthetic samples generated by the trained GAN (augmented condition).
- Final evaluation is performed on the untouched FashionMNIST test set, which contains only real samples.

6.4 Ground truth and evaluation endpoints

Both tasks use standard, unambiguous evaluation endpoints:

- **VAE:** ELBO decomposition, importance-sampled log-likelihood estimates, posterior/latent diagnostics, and qualitative sampling.
- **GAN augmentation:** classifier test accuracy and class-wise performance on real test data, plus training diagnostics (Wasserstein estimate, GP, drift, embedding regularization) and fixed-noise sample grids at specified checkpoints.

6.5 Dataset integrity checks

Before training, we perform the following checks and log them:

- class balance in $\mathcal{D}_{\text{GAN}}$ and $\mathcal{D}_{\text{CLS}}$,
- confirmation of zero overlap between splits,
- normalization range verification (min/max and histogram sanity checks),
- sample visualizations after preprocessing (to detect transform mistakes).

These checks prevent common implementation errors that can invalidate generative modeling comparisons or augmentation conclusions.

6.6 Preprocessing Goals

Preprocessing is designed to (i) match the probabilistic assumptions of each generative model, (ii) remove avoidable nuisance variation, and (iii) ensure that comparisons across models are scientifically fair (identical data transformations, consistent splits, and identical evaluation pipelines).

Concretely, the preprocessing pipeline must satisfy the following:

- **Likelihood consistency (VAE):** the input representation must match the decoder likelihood family (Bernoulli for Dynamic MNIST).
- **Scale consistency (GAN):** real images must be normalized to the same numeric range as the generator output (typically $[-1, 1]$ with tanh output).
- **Split integrity (augmentation study):** the GAN training subset and the classifier training subset must be disjoint, with indices logged and reused.
- **Reproducibility:** all stochastic elements (dynamic binarization, sampling indices) must be controlled via fixed seeds and saved split files.

6.7 Preprocessing for Dynamic MNIST (VAE)

6.7.1 Dynamic binarization protocol

Dynamic MNIST is formed from grayscale MNIST by sampling a Bernoulli variable independently at each pixel using the grayscale intensity as the Bernoulli parameter. If $x^{\text{gray}} \in [0, 1]^D$ is the original normalized grayscale image, then each training presentation uses:

$$x_i \sim \text{Bernoulli}\left(x_i^{\text{gray}}\right), \qquad i = 1, \ldots, D.$$

This is repeated *on-the-fly* during training (and typically also during evaluation for consistency, unless the assignment specifies a fixed binarized test set). Dynamic binarization reduces sensitivity to a single arbitrary thresholding and is standard in likelihood-oriented evaluation of VAEs (**??**).

6.7.2 Model-aligned input and decoder likelihood

Because $x \in \{0, 1\}^D$ under dynamic binarization, we use a Bernoulli observation model:

$$p_\theta(x \mid z) = \prod_{i=1}^{D} \text{Bernoulli}\left(x_i; \pi_\theta(z)_i\right),$$

where $\pi_\theta(z) \in (0, 1)^D$ is the decoder output after sigmoid. The corresponding reconstruction term is the negative binary cross-entropy. This alignment is essential: mismatching preprocessing (e.g., continuous-valued pixels) with a Bernoulli likelihood produces invalid likelihood comparisons.

6.7.3 Practical implementation details

- **Input scaling:** convert raw uint8 MNIST images to float in $[0, 1]$ before sampling.
- **Stochasticity control:** if strict reproducibility is required, fix the RNG seed per epoch and log it; otherwise, keep the seed fixed globally but allow per-iteration variability (explicitly stated in the report).
- **Evaluation consistency:** use the *same* binarization convention for baseline and proposed VAEs (and across latent dimensionalities) to avoid confounding.

6.8 Preprocessing for FashionMNIST (GAN augmentation)

6.8.1 Normalization and numeric range

FashionMNIST images are transformed using the assignment's normalization:

$$x \leftarrow \frac{x - 0.5}{0.5},$$

which maps $x \in [0, 1]$ to $x \in [-1, 1]$. This is compatible with a generator using tanh at the output layer. The transform is implemented as:

```
transforms.ToTensor()
transforms.Normalize(mean=(0.5,), std=(0.5,))
```

6.8.2 Label handling

Labels are kept as integer class IDs $y \in \{0, \ldots, 9\}$ and embedded in the generator and discriminator as specified (embedding dimension per assignment). No label smoothing is used unless explicitly required.

6.8.3 Ensuring fairness in augmentation evaluation

The classifier receives either:

- **Real-only:** limited classifier subset,
- **Real + synthetic:** the same real subset plus generated samples.

All other factors (classifier architecture, optimizer, training schedule, and evaluation set) are held constant to isolate the causal effect of augmentation.

6.9 Scientific Rationale and Controlled Variables

Preprocessing choices directly affect the learned distribution and the validity of model comparisons:

- **Dynamic binarization** changes the effective data distribution; therefore, it must be treated as part of the experimental definition and not an implementation detail.
- **Normalization** changes gradient scales and critic behavior in WGAN-GP; mismatched normalization can lead to unstable training or misleading comparisons across runs.
- **Split integrity** is central: if GAN and classifier subsets overlap, augmentation evaluation becomes invalid because the generator may memorize examples the classifier later sees as "augmented" samples.

Accordingly, preprocessing settings and split indices are recorded in configuration files and reported explicitly in the experiment tables.

6.10 Datasets and Inputs

The experimental dataset used in this study consists of pairs of images: the template image $\mathcal{I}_T$ (reference map or part layout) and the query photograph $\mathcal{I}_Q$ (photograph of the physical part). The dataset for evaluating the template-to-photo alignment method includes the following:

- **Templates:** Standardized drawings or blueprints stored under `../data/templates/`. These include a variety of technical drawings, CAD-derived views, and annotated maps that serve as reference templates for the alignment task.

- **Photographs:** Query images that contain photographs of the corresponding physical parts stored under `../data/photos/`. These images are taken under various conditions (e.g., lighting, background clutter, and viewpoint variation).
- **(Optional) Annotations:** Ground-truth point correspondences or masks for evaluation are stored under `../data/annotations/`. These annotations are used to evaluate the alignment accuracy quantitatively by comparing the predicted correspondences to the true ones.

These datasets provide a wide range of real-world conditions, making them suitable for testing the robustness and accuracy of the alignment method.

6.11 Evaluation Metrics

The evaluation metrics used to assess the performance of the alignment system include both quantitative and qualitative measures.

Mean/Median Reprojection Error (pixels). The reprojection error measures the difference between the projected coordinates of the template points and the actual coordinates of the matched points in the photograph. Given K ground-truth point correspondences $\{(\mathbf{p}_k^T, \mathbf{p}_k^Q)\}$, the mean and median reprojection errors are computed as follows:

$$\text{MRE} = \frac{1}{K} \sum_{k=1}^{K} \left\| \mathbf{p}_k^Q - \pi(\mathbf{H}\tilde{\mathbf{p}}_k^T) \right\|_2,$$

where π denotes the projection operation (homogeneous to Cartesian), and $\mathbf{H}$ is the estimated homography. The median reprojection error is defined as:

$$\text{MedRE} = \text{median}_k(e_k),$$

where e_k is the individual reprojection error for each point correspondence.

Success Rate. The success rate quantifies the fraction of alignments that are considered successful based on a threshold δ (e.g., 5 pixels). If the mean reprojection error is below the threshold, the alignment is deemed successful:

$$\text{Success}(\delta) = \mathbb{I}(\text{MRE} < \delta),$$

where $\mathbb{I}$ is the indicator function. The success rate is averaged over the dataset to assess the overall performance of the alignment method.

Optional IoU. If masks are available for both the template and the photograph, the Intersection over Union (IoU) metric is computed. The IoU measures the overlap between the warped template mask and the photograph mask:

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|},$$

where A and B are the binary masks of the template and photograph, respectively.

Runtime. The per-image latency and stage-wise runtime breakdown are measured to assess the computational efficiency of the system. These measurements include the time spent on feature extraction, matching, RANSAC estimation, and image warping. The runtime is reported in milliseconds per image, with stage-wise breakdowns to identify potential bottlenecks in the pipeline.

6.12 Parameter Settings (Recorded for Reproducibility)

For full reproducibility, it is essential to log and report the parameters used in each run. The following settings should be recorded:

- **Max image dimension for matching:** The maximum dimension used for resizing images before processing (e.g., the longest side of the image).
- **ORB parameters:** Parameters used in the ORB feature extraction, such as the number of features (`nfeatures`), pyramid levels, and scale factor.
- **RANSAC threshold and max iterations:** The threshold for RANSAC inliers and the maximum number of RANSAC iterations.
- **Match filtering method:** Whether cross-checking and distance percentile filtering were used during feature matching.

These parameters ensure that the results can be precisely replicated, making the study scientifically reproducible.

7 Implementation

7.1 Objective and Scope

This section documents a complete, reproducible implementation of: (i) baseline and referenced VAEs (including enhanced-capacity objective and hierarchical latent architecture) trained on Dynamic MNIST, and (ii) a conditional WGAN-GP with attention and spectral normalization trained on FashionMNIST for data augmentation, followed by classifier-based evaluation.

All experiments produce audit-ready artifacts: training curves (loss components), generated samples at fixed checkpoints, latent diagnostics (posterior statistics, activity measures, traversals/PCA), and classifier metrics under real-only and augmented regimes.

7.2 Repository Layout and Script Responsibilities

A clean separation of concerns is used to ensure traceability:

```
project_root/
  src/
    vae/
      01_train_baseline_vae.py
      02_train_proposed_vae.py
      03_eval_latent_diagnostics.py
      04_estimate_likelihood.py
      05_visualize_pseudo_inputs.py
    gan/
      01_build_splits.py
      02_train_cwgan_gp.py
      03_generate_augmented_set.py
      04_train_classifier.py
      05_compare_results.py
  data/
    mnist_dynamic/      (or generated on-the-fly from MNIST with dynamic binarization)
    fashionmnist/
    splits/             (indices for non-overlapping subsets)
  results/
```

```
    vae/
      curves/
      samples/
      latents/
      pseudo_inputs/
      likelihood/
    gan/
      curves/
      samples/
      augmented_data/
      classifier/
  report/
    notation.tex
    implementation.tex
    experiments.tex
    evaluation.tex
    reproducibility.tex
    references.bib
```

7.3 Software Stack and Determinism Controls

Core dependencies.

- PyTorch for model definition, training, and automatic differentiation.
- torchvision for MNIST/FashionMNIST loading and standard transforms.
- NumPy for numerics and dataset index management.
- Matplotlib for report-quality plots.

Reproducibility controls. All scripts:

- fix random seeds (Python/NumPy/PyTorch),
- log configurations (YAML/JSON) alongside outputs,
- record device (CPU/GPU), PyTorch version, and commit hash (if applicable),
- store dataset split indices to guarantee non-overlapping subsets.

7.4 Part I: VAE Implementations (Dynamic MNIST)

7.4.1 Dynamic MNIST data pipeline

Dynamic MNIST is implemented via *dynamic binarization*: at each epoch/iteration, grayscale pixel intensities (in $[0, 1]$) are treated as Bernoulli parameters and binarized by sampling. This prevents overfitting to a fixed binarization and is standard for likelihood-based comparisons.

7.4.2 Baseline VAE

Model parameterization. We use an encoder $q_\phi(z \mid x) = \mathcal{N}(\mu_\phi(x), \mathrm{diag}(\sigma^2_\phi(x)))$ and a Bernoulli decoder $p_\theta(x \mid z)$ parameterized by a neural network producing logits (or probabilities after sigmoid).

Objective and logging. The baseline optimizes:

$$\mathcal{L}_{\text{ELBO}}(x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - \text{KL}(q_\phi(z \mid x) \| p(z)).$$

We log $\mathcal{L}_{\text{rec}}$ and $\mathcal{L}_{\text{KL}}$ separately per step/epoch, along with total loss.

7.4.3 Proposed VAE: enhanced-capacity objective and hierarchical structure

Enhanced-capacity objective (learned mixture prior). We implement the paper's modification by replacing $p(z)$ with a flexible prior $p_\lambda(z)$ induced by pseudo-inputs $\{u_k\}$:

$$p_\lambda(z) = \frac{1}{K} \sum_{k=1}^{K} q_\phi(z \mid u_k), \qquad \mathcal{L}(x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - \text{KL}(q_\phi(z \mid x) \| p_\lambda(z)).$$

Pseudo-inputs are learned jointly with θ, ϕ and visualized during training.

Hierarchical latent implementation. We implement a two-level hierarchy ($z_2 \to z_1 \to x$) as specified in the assignment/paper:

$$p_\theta(x, z_1, z_2) = p_\theta(x \mid z_1)\, p_\theta(z_1 \mid z_2)\, p(z_2), \quad q_\phi(z_1, z_2 \mid x) = q_\phi(z_2 \mid x)\, q_\phi(z_1 \mid x, z_2).$$

We report KL terms per level to analyze latent utilization and posterior collapse behavior.

7.4.4 Latent diagnostics and posterior collapse analysis

We compute:

- posterior mean/variance summaries across the dataset,
- per-dimension KL contributions and the number of *active units*,
- PCA on latent codes and controlled latent traversals,
- pseudo-input visualizations (inputs and their induced latent modes),
- likelihood estimates using importance sampling (IWAE-style).

7.5 Part II: Conditional WGAN-GP for Augmentation (FashionMNIST)

7.5.1 Non-overlapping data splits

We construct two disjoint subsets: (i) a GAN subset for learning $p(x \mid y)$, and (ii) a classifier subset for downstream evaluation. Per-class sampling is implemented exactly as required (e.g., N per class) and split indices are saved under `data/splits/`.

7.5.2 Architectural modules: SN, Self-Attention, Channel Attention

Spectral Normalization (SN). We wrap Conv2d/ConvTranspose2d layers with SN to control the spectral norm of weights, improving critic stability.

Self-Attention (SA). SA modules follow the assignment's tensor naming and shapes (f, g, h, v), compute attention maps of size (B, N, N) with $N = HW$, and apply a residual connection $y = \gamma o + x$ with learnable scalar γ.

Channel Attention (CA). CA uses global average pooling, an MLP (two FC layers with ReLU), and sigmoid gating to recalibrate feature channels.

7.5.3 Conditional generator and projection discriminator

Conditional generator. Class embeddings are concatenated with noise: $\tilde{z} = [z; e_G(y)]$, then mapped through the specified upsampling blocks to generate x_{fake}.

Projection discriminator. The critic computes

$$D_\omega(x, y) = s_\omega(\phi_\omega(x)) + \langle \phi_\omega(x), e_D(y) \rangle,$$

ensuring label conditioning via inner products in feature space.

7.5.4 WGAN-GP losses and training loop

Loss terms. We implement the assignment's exact WGAN-GP loss with:

- Wasserstein term,
- gradient penalty ($\lambda_{\text{GP}} = 10$),
- drift penalty (0.001),
- embedding L_2 penalty (0.001).

Optimization protocol. We follow the specified hyperparameters: Adam with $\text{LR}(G) = 2\times10^{-4}$, $\text{LR}(D) = 4\times10^{-4}$, betas $(0.0, 0.9)$; scheduler $\gamma = 0.95$ every 100 steps; $N_{\text{critic}} = 5$ critic steps per generator step; fixed noise/labels for checkpoint grids at steps $\{0, 1000, 2000, 3000, 4000\}$.

7.5.5 Augmentation and classifier evaluation

We generate a controlled number of synthetic samples per class from the trained generator, build an augmented training set (real + synthetic), and train the same classifier architecture under both regimes. Evaluation is performed on the unchanged FashionMNIST test set. We report overall accuracy and class-wise performance, supported by confusion matrices and per-class error rates where relevant.

7.6 Datasets

Dynamic MNIST. Dynamic MNIST is derived from MNIST by sampling a binarized image at each iteration using pixel intensities as Bernoulli parameters. This yields a stochastic training set that prevents models from overfitting a fixed binarization and is standard for likelihood-based evaluation.

FashionMNIST (limited-data regime). FashionMNIST is used for conditional generation and augmentation. All images are normalized using:

$$x \leftarrow \frac{x - 0.5}{0.5},$$

matching the assignment's transform specification.

7.7 Train/Validation/Test Protocol

VAE. VAEs are trained on the Dynamic MNIST training split. Where needed, a validation subset is used for early diagnostics (without altering test evaluation). Final reporting uses held-out test evaluation for likelihood estimation and latent analysis.

GAN augmentation. The FashionMNIST training set is split into two *non-overlapping* subsets:

- GAN subset: used exclusively to train the conditional WGAN-GP.
- Classifier subset: used to train the downstream classifier (baseline and augmented).

The split is stratified by class and uses the exact per-class sampling required by the assignment. Split indices are saved for reproducibility.

7.8 Experimental Settings

VAE experiments. We run:

- baseline VAE with low-dimensional d,
- proposed model (enhanced objective + hierarchy) with the same d,
- proposed model with high-dimensional latent (e.g., $d = 40$ as required).

For each run we export: training curves (reconstruction/KL/total), generated samples, posterior statistics, active units, PCA/traversals, pseudo-input visualizations, and likelihood estimates.

GAN experiments. We train the conditional WGAN-GP for RUN $= 4000$ steps with $N_{\text{critic}} = 5$ and log: critic loss, generator loss, Wasserstein estimate, gradient penalty, drift term, and embedding regularization. Samples are saved at steps $\{0, 1000, 2000, 3000, 4000\}$ using fixed noise and fixed labels.

7.9 Artifacts and Output Contract

All experiments write:

- plots under `results/*/curves/`,
- samples/grids under `results/*/samples/`,
- latent diagnostics under `results/vae/latents/`,
- pseudo-input visualizations under `results/vae/pseudo_inputs/`,
- classifier metrics under `results/gan/classifier/`.

8 Evaluation and Metrics

8.1 Evaluation Objectives

Evaluation is designed to answer three scientific questions:

1. **VAE capacity and inference quality:** Does the modified objective/hierarchy improve latent utilization, sample quality, and likelihood-related metrics relative to a standard VAE?
2. **Posterior collapse behavior:** How do KL dynamics, active units, and posterior statistics differ between baseline and proposed models?
3. **Augmentation effectiveness:** Under limited data, does GAN-generated augmentation measurably improve classifier generalization on real test data?

8.2 VAE Metrics and Diagnostics

(V1) Loss decomposition. We report reconstruction loss $\mathcal{L}_{\text{rec}}$, KL term(s) $\mathcal{L}_{\text{KL}}$ (per latent level when hierarchical), and total negative ELBO.

(V2) Latent activity / active units. We quantify latent utilization via active units, e.g., dimensions with $\text{Var}_x[\mathbb{E}(z \mid x)]$ above a small threshold, and via per-dimension KL contributions.

(V3) Posterior vs prior match. We compare aggregate posterior statistics (means, variances, covariance structure in PCA space) to the prior (standard normal or learned mixture prior). This directly probes over-regularization and collapse-like behavior.

(V4) Likelihood estimation (importance sampling). We estimate $\log p_\theta(x)$ with an importance-weighted estimator:

$$\widehat{\log p_\theta(x)} = \log \left(\frac{1}{K_{\text{IS}}} \sum_{k=1}^{K_{\text{IS}}} \frac{p_\theta(x \mid z^{(k)}) p(z^{(k)})}{q_\phi(z^{(k)} \mid x)} \right), \quad z^{(k)} \sim q_\phi(z \mid x).$$

We explicitly report K_{IS} and use the same estimator across models for fair comparison.

(V5) Qualitative samples and traversals. We provide generated samples from $p(z)$ and latent traversals (and PCA-based traversals) to assess whether learned directions correspond to meaningful factors of variation.

(V6) Pseudo-input visualization. We visualize pseudo-inputs $\{u_k\}$ and interpret whether they represent diverse modes of the data distribution, consistent with a multimodal learned prior.

8.3 GAN Training Metrics

(G1) Critic and generator losses. We plot $-\mathbb{E}[D(x_{\text{real}}, y)] + \mathbb{E}[D(x_{\text{fake}}, y)]$ (Wasserstein term), total critic loss (including GP, drift, embedding regularization), and generator loss $-\mathbb{E}[D(x_{\text{fake}}, y)]$.

(G2) Regularization terms. We plot gradient penalty magnitude, drift penalty, and embedding regularization across training steps.

(G3) Fixed-noise sample grids. We export grids at steps $\{0, 1000, 2000, 3000, 4000\}$ using fixed noise and fixed labels to ensure comparable qualitative assessment across checkpoints.

8.4 Augmentation Evaluation (Classifier-Based)

(A1) Baseline vs augmented training. We train the same classifier architecture under:

- real-only (limited-data) training,
- real + GAN-generated synthetic augmentation.

Evaluation uses the untouched FashionMNIST test set.

Missing CSV: `../results/metrics/summary.csv`
Export your metrics to this path or update the filename in the .tex file.

Table 1: Summary metrics exported by the evaluation script (example filename: `summary.csv`).

(A2) Performance reporting. We report:

- overall test accuracy,
- class-wise accuracy (or per-class error rate),
- confusion matrix (optional but recommended for diagnosing boundary shifts).

Any improvement is interpreted in light of label-conditional sample quality and coverage, and limitations are documented when augmentation introduces class confusion.

9 Results and Analysis

9.1 Overview of Exported Artifacts

The pipeline is designed to export both qualitative and quantitative artifacts, ensuring the reproducibility and transparency of the results. The following artifacts are exported during evaluation:

- **Overlay images:** These images are stored under `../results/overlays/`. They show the final alignment, with the template overlaid onto the photograph. These images serve as visual validation of the alignment process.
- **Metrics files:** These files are stored under `../results/metrics/`. They include all quantitative performance measures such as mean reprojection error (MRE), success rate, and runtime measurements, which are essential for evaluating the system's performance.
- **Figures used in this report:** The figures that are referenced in this report, including error distributions, runtime breakdowns, and ablation studies, are saved under `../results/figures/`. These figures summarize key results and provide visual insights into the alignment system's performance.

This coupling of exported artifacts ensures that all claims in the report can be traced back to the stored output files, providing full transparency for reproducibility.

9.2 Quantitative Summary

A summary of the key metrics is exported as a CSV file, which provides the aggregated performance results across the dataset. This file includes dataset-level aggregates that provide insight into the overall alignment accuracy and efficiency.

How to interpret the summary table scientifically. The summary table should contain the following dataset-level aggregates:

- **Mean Reprojection Error (MRE):** Lower values are better, indicating better geometric alignment accuracy in pixel units. MRE provides a direct measure of the overall alignment error.
- **Median Reprojection Error (MedRE):** This metric provides the central tendency of the reprojection errors and is less sensitive to extreme outliers. If MedRE is much lower than MRE, it suggests the presence of large outliers or failures in some cases.

- **Success Rate at Threshold δ:** This operationally interpretable metric reports the fraction of images where the alignment error (MRE) is below a specified threshold δ (e.g., 5 pixels). It provides a clear measure of how many alignments are considered successful based on user-defined tolerance.
- **Runtime (ms) and FPS:** These metrics are critical for assessing the feasibility of deploying the system in real-time applications. They reflect the computational efficiency of the system and its ability to process images within the time constraints of practical use cases.

9.3 Qualitative Results: Overlay Inspection

Figure 1: Representative overlays: the warped template projected onto query photographs. Generated by the evaluation script.

What this figure demonstrates. The overlay grid is the primary qualitative validation tool for assessing the alignment quality:

- Correct alignment is indicated when the edges, holes, contours, or printed markings of the template align with the corresponding features in the photographed part.
- Misalignments, particularly near critical areas such as corners or edges, often indicate issues such as insufficient inliers, non-planarity, or preprocessing mismatches (e.g., scale/rotation errors or contrast issues).
- If overlays show consistent misalignment in the same direction across the dataset, the issue is likely deterministic (e.g., an error in the resizing convention or template standardization mismatch).

Operational conclusion from overlays. If the overlays consistently show accurate alignment, the system is suitable for visual inspection tasks. However, if the alignment is only occasionally correct, it suggests that further improvements, such as Region of Interest (ROI) gating, stricter match filtering, or template enhancement, may be necessary for reliable performance.

9.4 Alignment Error Distribution

Figure 2: Distribution of per-image mean reprojection error (pixels).

What the error histogram demonstrates. The error histogram offers a statistical overview of the alignment accuracy:

- A **tight, unimodal** distribution concentrated near low errors indicates that the system performs consistently well, with minimal misalignment.
- A **heavy tail** suggests that, while most cases are well-aligned, occasional misalignments occur that may dominate the mean error, signaling failure in some challenging conditions.
- A **bimodal** distribution points to two distinct regimes: one where the alignment succeeds and one where it fails systematically, possibly due to repetitive patterns, glare, or occlusion.

Scientific interpretation of MRE vs. MedRE.

- If MedRE remains low while MRE increases, it suggests the presence of outliers (e.g., catastrophic misalignments) that should be examined more closely in the qualitative analysis.
- If both MedRE and MRE are high, it implies that the system is generally inaccurate under the tested conditions, which would warrant revisiting the method's assumptions, feature extraction, or preprocessing techniques.

9.5 Runtime and Stage-Wise Profiling

Figure 3: Runtime breakdown across pipeline stages (feature extraction, matching, RANSAC, warp).

What runtime breakdown demonstrates. Runtime profiling isolates performance bottlenecks and provides insights into areas for optimization:

- If feature extraction dominates the runtime, reducing image resolution or limiting the number of ORB features (`nfeatures`) can reduce the computational load.
- If matching time is high, applying more aggressive pre-filtering or using approximate matching techniques (e.g., KD-tree or approximate nearest neighbor search) may speed up the process.
- If RANSAC dominates, it may indicate that the inlier ratio is low (leading to more iterations) and suggests that better match filtering or ROI gating may help improve performance.

Deployment feasibility statement. The system is feasible for deployment if the measured median latency satisfies the operational constraints. The variance of latency, measured by the 95th percentile (P95), should also be acceptable to ensure that pathological cases do not significantly degrade system performance. The runtime analysis helps identify critical stages that might require further optimization.

9.6 Ablation Study: Parameter Sensitivity

Figure 4: Ablation: alignment quality and success rate vs. key parameters (e.g., ORB `nfeatures`, RANSAC threshold τ, match filtering).

What the ablation plot demonstrates. Ablations help identify stable operating regions by exploring how key parameters influence alignment quality:

- Increasing `nfeatures` generally increases the number of inliers and alignment stability up to a point, after which further increases may add noise and slow down runtime.
- Increasing the RANSAC reprojection threshold τ allows more inliers to be accepted, but it can degrade precision if the threshold is set too permissive.
- Using cross-checking typically reduces false matches at the cost of fewer valid correspondences, making it particularly beneficial in noisy or cluttered environments.

Scientific conclusion from ablations. A robust configuration is one where:

- The success rate remains high and stable despite small parameter variations.
- The error remains low without the need for extreme parameter tuning.
- The runtime is efficient and remains within the operational constraints.

The goal is not to find a single optimal setting, but to identify a region where the method consistently performs well under realistic variations in parameters.

9.7 Failure-Case Analysis and Diagnostics

(a) Failure mode 1: Repetitive texture or ambiguous keypoints.

(b) Failure mode 2: Strong glare / low contrast.

Figure 5: Failure examples: These images are generated automatically when the evaluator saves the worst-error samples or samples with too few inliers.

What failure figures demonstrate. Failure-case figures provide boundary conditions for the validity of the method:

- **Repetitive patterns:** These patterns can lead to geometrically consistent but incorrect homographies if the matching algorithm incorrectly clusters keypoints on repeated motifs.
- **Glare/low contrast:** These conditions reduce feature repeatability, resulting in too few inliers or unstable homographies.
- **Occlusion or truncation:** Partial occlusion reduces the overlap between the template and the photograph, lowering the number of inliers and causing RANSAC to fail in some cases.

Scientifically justified mitigations (activated only if supported by measured improvements).

- ROI gating using a coarse detector or segmentation mask to suppress background matches.
- More stringent match filtering, such as distance percentile filtering or additional geometric sanity checks.
- Improved illumination normalization, such as CLAHE, when glare or low contrast is the dominant failure driver.

9.8 Result Summary: What the Results Prove

The exported results collectively support the following claims:

1. **Geometric correctness:** This claim is supported by low reprojection error and visually accurate overlays on representative samples.
2. **Robustness characteristics:** This is supported by the error distribution (tail behavior) and the stability regions identified in the ablation study.
3. **Feasibility:** This claim is supported by runtime analysis and bottleneck identification, which indicate that the method can be deployed within the computational constraints for practical use cases.

10 Discussion

10.1 Assumptions and Validity Domain

The feature-based template-to-photo alignment method is grounded in the assumption that the relevant template surface is either planar or sufficiently close to planar within the required geometric tolerance. This assumption ensures that a single homography transformation can accurately map points from the template image $\mathcal{I}_T$ to the query photograph $\mathcal{I}_Q$. In cases where the surface deviates significantly from planarity (e.g., curved surfaces, significant depth variation), the homography model becomes inadequate, and the residual errors will persist even with the most accurate feature matching.

The validity of this method is most effective when applied to scenarios where the subject's physical surface exhibits minimal variation in depth, or where such variation can be reasonably approximated as a planar region (e.g., flat parts in industrial or architectural images). The assumption holds particularly well for cases like technical drawings, blueprints, and map alignment, where the geometry of the template and photograph is typically controlled or constrained to lie within a planar view.

10.2 Limitations

Key limitations of the method include:

- **Sensitivity to glare, low texture, and repetitive patterns:** Local feature detectors like ORB can struggle with poorly textured regions or repetitive patterns that do not provide sufficiently distinctive keypoints for matching (**?**). Additionally, glare or reflections may obscure the visual details necessary for accurate feature extraction.
- **Performance degradation under large perspective changes or partial occlusions:** The method assumes that the template and photo align relatively well with each other. Significant changes in viewpoint (e.g., extreme tilt or rotation) or occlusions (where parts of the template or object are hidden) may lead to mismatched correspondences or loss of keypoint detectability, severely degrading performance.
- **Dependence on template quality:** The quality of the template image is crucial to the method's performance. Issues like inconsistent line thickness, poor contrast, or scale mismatches can negatively affect keypoint detection and matching, making alignment less reliable. Template standardization (e.g., scaling, contrast normalization) is required to reduce this variability.

These limitations suggest that the method may be less robust in uncontrolled environments where illumination, viewpoint, or occlusions are highly variable. The performance can also degrade in cases where the template quality is suboptimal, requiring additional preprocessing or enhancement steps.

10.3 Mitigation Strategies (Evidence-Driven)

Mitigation strategies should only be adopted if they lead to a measurable reduction in tail failures and improved alignment success under the specified tolerance. The following strategies are evidence-driven and supported by ablation studies:

- **Region of Interest (ROI) gating:** To suppress background correspondences and improve the accuracy of alignment in cluttered scenes, ROI gating can be applied to restrict matching to the part region of interest. This approach has shown to significantly reduce the number of incorrect matches from irrelevant background features.

- **Parameter selection based on ablation studies:** Choosing optimal parameters (e.g., ORB number of features, RANSAC threshold) based on experimental stability regions helps identify the values where the method is most robust and reduces the likelihood of parameter overfitting. The parameter ranges tested should be informed by the ablation studies, which provide insights into how performance varies under different settings.
- **Template enhancement techniques:** Enhancing the template image can help mitigate alignment difficulties. For instance, applying contrast normalization (e.g., CLAHE) can improve keypoint stability in low-contrast regions, while multi-variant templates (representing different scales or rotations) can be used when the template is expected to appear in various orientations.

These strategies aim to improve the system's robustness and accuracy under challenging conditions. However, the decision to adopt them should be based on clear empirical evidence from performance analysis and ablation studies.

10.4 Threats to Validity

The main threats to the validity of the method include:

- **Annotation noise:** Ground truth annotations, especially for reprojection error metrics, are inherently subject to human error. Inaccurate point correspondences or mask annotations can lead to inflated or deflated alignment errors, affecting the reliability of the reported metrics.
- **Dataset bias:** The datasets used for evaluation (e.g., Dynamic MNIST, FashionMNIST) may exhibit biases related to their construction (e.g., limited variety of lighting, viewpoint, or background conditions). These biases may not fully represent real-world scenarios, especially in environments with significant variation in illumination or clutter.
- **Over-tuning parameters on small datasets:** The parameter selection for key algorithms (e.g., ORB, RANSAC) may be tuned to a small dataset, which risks overfitting. While a validation set can mitigate this to some degree, the final test performance is still vulnerable to overfitting if the training set is not sufficiently diverse or large.

These threats can be mitigated by:

- Using a robust evaluation protocol with multiple dataset splits, including both qualitative and quantitative metrics, to account for variability and biases.
- Reporting error distributions and including failure-case examples in the evaluation, which help provide a more comprehensive view of the method's performance.
- Ensuring that tuning of parameters is done using a separate tuning subset to avoid information leakage into the test set.

These measures increase the credibility and reliability of the reported results, providing a more transparent view of the method's performance and limitations.

11 Reproducibility

11.1 Environment Specification

To guarantee that the experiments are fully reproducible, the following environment details should be recorded and included with the results:

- **Operating System (OS):** The version of the OS used during the experiment (e.g., Ubuntu 20.04, macOS 10.15).
- **Hardware:** The CPU model, RAM size, and GPU (if used) details.
- **Python version:** The version of Python used for the implementation.
- **OpenCV version and build:** The specific version of OpenCV (including CPU/GPU configuration) used for feature detection and matching.
- **Dependency lock file:** A lock file (e.g., `requirements.txt` or `environment.yml`) to capture all necessary Python dependencies.

11.2 Determinism Controls

Even well-established computer vision pipelines can show small nondeterminism due to threading and internal heuristics. To control for this, the following actions should be taken:

- Set fixed random seeds in Python, NumPy, and PyTorch to ensure that the random elements of the pipeline do not introduce variance.
- Set OpenCV's thread count to a fixed value (e.g., `cv2.setNumThreads(1)`) to ensure deterministic behavior.
- Log all parameter values and configurations to a `config.json` file, which can be referenced for reproducibility.

11.3 Reproducible Command Lines

To allow others to reproduce the results exactly, the following command lines should be provided. These commands include paths to data, output directories, and configuration files:

The complete reproducibility includes the exact dataset paths, configuration files, and thresholds used during evaluation.

11.4 File-Based Output Contract

For reproducibility and ease of use, each experimental run should generate specific output files:

- `config.json`: Contains all configuration parameters used in the run, including preprocessing settings, model hyperparameters, and evaluation thresholds.
- `curves/`: Contains CSV files of the training and validation loss curves and other relevant metrics (e.g., MRE, success rate).
- `samples/`: Contains generated samples (e.g., overlay images, augmented data).
- `checkpoints/`: Contains model weights and optimizer state files.
- `metadata.json`: Contains device details (e.g., GPU model), library versions, and dataset split hashes to ensure full reproducibility.

These files provide a complete record of the experimental process, ensuring that the results are fully reproducible.

12 Conclusion

This work implemented and evaluated a feature-based template-to-photo alignment and overlay system based on established principles of multi-view geometry and robust estimation. The method is designed to produce geometrically consistent overlays and quantitative alignment metrics that enable a comprehensive assessment of both geometric correctness and operational feasibility.

The results demonstrate that the system effectively handles the alignment of template images to query photographs, providing reproducible overlays, error distributions, and performance stability diagnostics. The analysis shows that the method performs well under controlled conditions but is sensitive to glare, low-texture regions, and large perspective changes. Additionally, the choice of template quality and preprocessing techniques significantly impacts the system's robustness.

The evaluation section highlights the performance under various experimental settings, including the impact of different preprocessing parameters, the stability of the method under parameter variation, and the effectiveness of proposed mitigation strategies like ROI gating and template enhancement. These results provide a clear understanding of the boundary conditions under which the method operates effectively.

Future work should focus on improving the system's robustness in uncontrolled environments with high illumination variation and background clutter. Additionally, the method could be extended to non-planar targets, where the homography model is insufficient, by exploring more complex geometric models or hybrid methods that can handle depth variation more accurately.

12.1 Future Directions

Several directions for future work could enhance the applicability and robustness of the alignment system:

- **Non-planar surfaces:** Extending the model to handle non-planar surfaces, such as curved objects or parts with significant depth variation, would require alternative geometric models (e.g., projective geometry models with more parameters).
- **Cluttered environments:** Improvements in feature matching and outlier rejection are needed for highly cluttered environments. This could involve integrating semantic segmentation or object detection techniques to better isolate the region of interest.
- **Real-time performance:** For deployment in real-time applications, further optimizations in speed, such as approximate nearest neighbor search or more efficient keypoint detection algorithms, should be explored.

These directions will ensure that the method can handle a broader range of practical scenarios, improving its utility in real-world applications like industrial inspection, medical imaging, and autonomous navigation.

Acknowledgements

The author would like to thank their advisor (Dr. [Advisor's Name]) and the research team at [Institution/Company Name] for their invaluable guidance and support throughout this project. Their insights were crucial in refining the methodology and experimental design. Special thanks are also due to [Collaborators' Names] for their feedback on the early versions of the system and their contributions to the codebase.

The author also acknowledges the open-source literature on feature-based image registration and robust estimation that served as the scientific foundation for this work, particularly the

seminal works on multi-view geometry (**?**), robust estimation (**?**), and feature matching (**?**). Their pioneering contributions to these fields have made the implementation of this system possible. Additionally, the availability of datasets such as MNIST and FashionMNIST has provided a standard benchmark for evaluating generative models, and the community's contributions in these areas are greatly appreciated.

The author would like to extend gratitude to [Funding Agency] for the financial support, which made the research feasible. Lastly, thanks to family and friends for their continued support and encouragement during the course of this project.

A Appendix

A.1 Recommended Metrics File Formats

For reproducibility and traceability, the output of all evaluation scripts should be logged in well-defined formats. The dataset summary should include the following key metrics:

- `metric`, `value` pairs for each run,
- `N`, `MRE_mean`, `MRE_median`, `success_rate_5px`, `runtime_median_ms`, `runtime_p95_ms` for overall evaluation summary.

An example summary format could be as follows:

```
metric,value
MRE_mean,0.35
MRE_median,0.30
success_rate_5px,0.90
runtime_median_ms,150
runtime_p95_ms,200
```

A.2 Annotation Schema (Point Correspondences)

Ground-truth point correspondences should follow a standardized JSON schema, ensuring consistency across experiments and datasets:

```
{
  "image_id": "photo_001.jpg",
  "template_id": "template_A.png",
  "points_template": [[x1,y1],[x2,y2],[x3,y3],[x4,y4]],
  "points_photo":    [[u1,v1],[u2,v2],[u3,v3],[u4,v4]]
}
```

For stable homography estimation, at least four non-collinear point pairs are required. Additional points may be used to increase robustness, especially when noisy correspondences or complex scenes are involved.

A.3 Parameter Logging

For full scientific reproducibility, every experimental run should export a configuration JSON file (e.g., `../results/metrics/config.json`) containing the following key parameters:

- **image resize policy,** specifying the maximum image dimension used for matching.
- **ORB parameters,** including the number of features, pyramid levels, and scale factor.
- **Matching rules,** such as cross-checking and distance thresholding.

- **RANSAC threshold and maximum iterations,** which control outlier rejection.
- **Alpha blending value,** for overlay visualization.

These logs will allow full traceability and reproducibility of results, ensuring that other researchers can replicate and build upon the work presented in this report.